

Andrew Long
Prof Jon McGown
CSIS-2430-501
DATE

Programming Project 1

For our first programming assignment of the semester we took a look and did some experimentation with sorting algorithms. A sorting algorithm is an algorithm that takes a collection of items and orders them in a specified sequence. These algorithms are important as they are used heavily in the field of computer science. Online stores use these to sort their catalogs, and their customers can even customize these to tailor for items they are looking for. Taking a deeper look into these algorithms and comparing the results is a good way to understand scale. As we will go over in the future parts of this paper, you can see that some of these algorithms work much faster and efficiently than some of the others, and as the data set they are sorting grows that speed and efficiency become all the more important.

My program works by creating multiple arrays of integers using java's random function, the arrays are created using specified sizes which are found in my sizes array in my main method. Each randomly generated array is then passed into four separate sorting algorithms, each comparison in each algorithm is counted, then when all the arrays are sorted we are presented with each array sorted and the information of how many comparisons were made to get things in the correct order.

For merge sort, the program begins by examining the length of the array, and splitting it into two separate arrays in the middle by taking its length and dividing by two. Two new arrays are created, left and right. This process then continues on recursively, dividing each half into more halves until all the elements of the original array have been isolated. Once we have reached the point where all of our array has been broken down into individual elements, a separate helper method is called which we use to recombine the arrays. The way we merge these elements is by comparing the first unmerged element of the left and right halves and placing the smaller of the two values into the output array. This continues until all elements from both halves have been merged back into a single sorted array. During each comparison we are incrementing a counter to keep track of these comparisons. Once the program compares the base elements and combines them into groups of two, it loops through and compares the elements left to right with the elements of another group left to right. This continues until we are left with a single sorted array.

The quick sort algorithm calls for a pivot, we used the last item in the array as our pivot point.